MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Trust-driven adaptive safety for Autonomous Vehicles

Master's Thesis

BC. ALEXANDR ČELAKOVSKÝ

Brno, Spring 2026

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Trust-driven adaptive safety for Autonomous Vehicles

Master's Thesis

BC. ALEXANDR ČELAKOVSKÝ

Advisor: Ing. Dávid Halász

Department of Computer Systems and Communications

Brno, Spring 2026


FACULTAS ARTIS INFORMATICÆ
UNIVERSITAS MASARYKIANA

Declaration

I declare that this thesis is my original work and that I prepared it independently. During the preparation of this thesis, I used ChatGPT as an artificial intelligence tool for programming assistance, stylistic editing, grammar checking, and code review support. I used this tool in accordance with academic integrity principles. I checked all generated suggestions and take full responsibility for the content of this thesis. All sources and literature used or referred to in this work are cited with full references.

Bc. Alexandr Čelakovský

Advisor: Ing. Dávid Halász

Acknowledgements

I would like to thank my family and friends for their long-term support. My thanks also go to my supervisor, Ing. Dávid Halász, for his guidance and consultations, and to Radovan Hančuľák for his help with this work.

Abstract

This thesis examines trust as an input for adaptive safety in autonomous vehicle ecosystems. The implementation is built in SUMO and TraCI and follows a modular trust-driven design. It combines observer-relative trust, local knowledge exchange, digital twin reasoning, and explicit safety adaptation. Each vehicle evaluates nearby actors from direct observation and weighted reputation and the resulting trust values affect following distance and crossroad behavior. Results from a complex interaction scenario and a highway benchmark indicate that the trust-driven approach can improve traffic efficiency and smoothness compared with the earlier adaptive variant, while keeping the design modular.

Keywords

autonomous vehicles, adaptive safety, trust management, SUMO, TraCI, Python

Contents

Introduction **1**

1 Background **3**
1.1 Autonomous Vehicles . . . 3
1.2 Autonomous Vehicle Ecosystems . . . 3
1.3 Previous Work . . . 4
1.4 Trust and Reputation . . . 5
1.5 Dynamic and Observer-relative Trust . . . 6
1.6 Local Knowledge Exchange . . . 7
1.7 Digital Twin . . . 8
1.8 Decision Rules . . . 8

2 Analysis and Planning **10**
2.1 SUMO . . . 10
2.2 TraCI . . . 11
2.3 Chosen Metrics . . . 12
2.3.1 Traffic Flow Metrics . . . 12
2.3.2 Smoothness and Stability Metrics . . . 13
2.3.3 Trust and Decision Outputs . . . 13
2.4 Simulation Scenarios . . . 14
2.5 Compared Trust Aware Methods . . . 15

3 Implementation **17**
3.1 Codebase Overview . . . 17
3.2 Runtime Loop . . . 18
3.3 Core Components . . . 19
3.4 Trust Storage and Update Logic . . . 22
3.5 Local Interaction and Knowledge Exchange . . . 26
3.6 Digital Twin Based Evaluation . . . 29
3.7 Safety Adaptation . . . 30
3.7.1 Following Distance . . . 31
3.7.2 Crossroad Behavior . . . 32
3.8 Policy Parameters and Exported Outputs . . . 34

4 Evaluation **37**
4.1 Evaluation Workflow . . . 37

4.2 Evaluation Scenarios 37
4.3 Current Results 38
4.4 Trust and Decision Outputs 40
4.5 Malicious Actors 42

5 Conclusion **45**

Bibliography **47**

A Electronic Attachments **50**

List of Tables

2.1 Main SUMO-related artifacts used by the simulation workflow. 11
4.1 Comparison across 50 runs on the complex scenario. . . . 39
4.2 Comparison across 50 runs on `simulation-complex_own_routes`. 39
4.3 Comparison across 50 runs on `simulation_highway_2`. . . 40
4.4 Malicious highway traffic results. 42
4.5 Malicious highway warning results. 43

List of Figures

1.1 Structured illustration of trust-based adaptation [2]. . . . 9
2.1 SUMO representation of the `simulation-complex` road network used as the primary interaction-rich scenario. . . 14
3.1 Runtime loop of the trust-driven simulation script. Each step observes vehicle state, updates trust, applies adaptation, and records metrics. 19
3.2 High-level trust architecture [1]. 21
3.3 Trust update flow used during one simulation step. 21
3.4 Spatial bucketing. 27
3.5 Spatial bucketing implementation. 27
3.6 Local knowledge exchange. 28
3.7 Example of nearby vehicles meeting in the simulation, triggering local trust exchange and direct trust evaluation. 28
3.8 Nearby vehicle encounter. 29
3.9 Digital twin based evaluation. 30
3.10 Intersection queue detection and evaluation. 31
3.11 Highway following distance conditions. 32
3.12 Crossroad speed adaptation. 33
3.13 Example of vehicles approaching an intersection, where the crossroad trust policy decides whether flow is preserved or caution is increased. 33
4.1 SUMO representation of the complex scenario with own routes. 38
4.2 Average final trust assigned by normal vehicles after meetings in the malicious highway scenario. Here, n is the number of met observer-target pairs in each group. 44

Introduction

Automated driving technology is increasingly present in everyday traffic, even when full vehicle autonomy remains limited. Modern vehicles can assist with speed control, lane keeping, braking, and other driving tasks, but they still operate in environments shaped by the behavior of surrounding actors. Safety then depends both on how well a vehicle controls itself and on how it reacts to nearby vehicles whose future behavior is uncertain.

The uncertainty is local and practical rather than only theoretical. A vehicle may know its own speed, lane, route, and distance to surrounding vehicles, but it cannot know with certainty whether a nearby vehicle will keep a stable speed, brake unexpectedly, approach an intersection cautiously, or disturb the traffic flow. These nearby traffic situations matter because small local decisions can propagate through the traffic stream as unnecessary braking, larger gaps, or unstable travel times.

A uniformly conservative safety policy can respond to this uncertainty by keeping large margins in most situations. Such behavior is defensible from a safety perspective, but it can also reduce traffic smoothness and throughput. A more permissive policy has the opposite weakness. It may preserve flow, but it gives too little margin around actors whose behavior is unreliable. The problem is not whether caution is needed, but how caution should change when the reliability of nearby vehicles changes.

The proposed model uses trust to guide this adjustment at runtime. Trust is not treated as a global label attached to a vehicle. Instead, it is an observer-relative estimate of how reliably one vehicle expects another nearby vehicle to behave in the current traffic context. Low trust can justify stronger caution, while higher trust can support smoother behavior when the surrounding vehicle has behaved consistently.

The work focuses on a trust-driven adaptive safety layer for simulated autonomous vehicle traffic. The design combines observer-relative trust, local knowledge exchange, comparison of expected and observed behavior through digital twins, and explicit decision rules. These mechanisms adapt selected safety behavior, especially following distance and vehicle behavior near crossroads.

The design is grounded in research by the trust team of the Laboratory of Software Architectures and Information Systems at Masaryk University's Faculty of Informatics [1, 2]. This research provides the main architectural direction by separating trust management, digital twin handling, scenario generation, simulation, and comparison of vehicle behavior. It also supports the use of explicit decision logic for adapting safety behavior with trust.

The main objective of this thesis is to design and evaluate a dynamic trust-driven approach that can adapt vehicle behavior during simulation. The research problem is how trust can be incorporated into safety decisions in a way that remains observer-relative, locally exchanged, explainable, and practical to run, while improving traffic flow and smoothness rather than only increasing caution.

The approach is implemented and evaluated in SUMO through TraCI. It is compared with the adaptive variant across many simulation runs using metrics related to traffic flow, smoothness, and safety. The evaluation focuses on whether the proposed trust-driven method improves traffic performance while preserving safety.

The rest of the text is structured as follows. Chapter 1 introduces the background on autonomous vehicles, trust, local knowledge exchange, and explicit decision rules. Chapter 2 explains the simulation environment, metrics, scenarios, and compared methods. Chapter 3 describes the implemented trust-driven architecture and its integration into the simulation project. Chapter 4 presents the results and chapter 5 summarizes the conclusions and outlines future work.

1 Background

1.1 Autonomous Vehicles

Autonomous vehicles are vehicles that use software, sensors, and control systems to perform driving tasks with limited or no direct human input. They observe the road around them, interpret the current traffic situation, and select actions such as maintaining speed, braking, steering, or keeping a safe distance from other vehicles.

In practice, vehicle automation exists on a spectrum. Many modern vehicles already include assistance functions such as adaptive cruise control, lane keeping, emergency braking, or parking assistance. These systems do not make the vehicle fully autonomous, because the human driver usually remains responsible for supervision. More advanced autonomous vehicles combine several such capabilities and make more driving decisions through software.

For this thesis, the relevant property of an autonomous vehicle is not its exact automation level, but the fact that part of its behavior is controlled by software and can be modeled, modified, and evaluated. A simulated vehicle can be given rules for following, slowing down, reacting near intersections, or communicating with nearby vehicles. These rules are simpler than a real autonomous driving stack, but they are sufficient for studying how local decision-making changes traffic behavior.

In a traffic simulation, autonomous vehicles can be treated as agents. Each vehicle can keep its own state, observe nearby actors, and make decisions from incomplete local information. When many such vehicles interact, small local rules can influence broader traffic outcomes such as travel time, throughput, smoothness, and the number of cautious maneuvers. The model in this thesis operates at this level.

1.2 Autonomous Vehicle Ecosystems

For this thesis, an autonomous vehicle ecosystem means the set of vehicles that share the same simulated road space and influence one another during a run. A vehicle does not decide in isolation. Its next

safe speed, gap, and intersection behavior depend on nearby actors, on what it has already observed, and on the trust knowledge it has received through local encounters. The relevant variation comes from route choices, current traffic state, trust histories, and policy decisions made by individual vehicles.

Research on autonomous ecosystems is useful because it treats autonomous actors as participants in a shared environment rather than as independent products [3, 4]. In this thesis, that view is narrowed to a simulation question. Trust is useful only if it changes a vehicle's local behavior. A value stored in a table but never used for following or crossroad decisions would not affect the ecosystem at all.

1.3 Previous Work

The work continues research carried out in the Laboratory of Software Architectures and Information Systems at Masaryk University's Faculty of Informatics on autonomous vehicle ecosystems and adaptive safety. The first directly related work is Richard Plesník's simulation thesis [5], which implemented and evaluated a fairness incentivization mechanism in SUMO. That work focused on improving traffic behavior through local cooperation between vehicles. The present thesis does not use the same token mechanism, but it builds on the same idea that local decisions made by individual vehicles can affect system-level traffic quality.

The second directly related work is Radovan Hančuľák's adaptive safety thesis [6]. That thesis introduced trust into adaptive safety behavior for autonomous vehicle simulation and studied how trust can affect vehicle behavior. The focus remains on trust-driven safety adaptation, but the model is extended in several directions. Trust is observer-relative, knowledge exchange is restricted to nearby encounters, recommendations are weighted by trust in the recommender, and trust is updated during runtime from observed behavior. This differs from the earlier static use of trust, where trust values were assigned before the simulation rather than changed by vehicle interactions.

The implementation also follows a broader research direction in which trust management, digital twin handling, scenario generation, simulation, and comparison are separated into distinct responsibilities

[1]. Here, that architectural idea is applied to simulated autonomous vehicle traffic in SUMO and TraCI.

The digital twin part is also related to work outside the traffic domain. Prior work on digital twins for autonomous drones describes runtime representations that support compliance checking, coordination, analysis, and trust in autonomous ecosystems [7]. The same work also describes trust assessment through declared behavior in a digital twin and real-time observation of whether the agent follows that behavior. Only this general idea is transferred to simulated vehicles. The direct trust calculation uses traffic signals such as lane, speed, acceleration, leader context, and intersection context, but it does not use the drone metrics from that work.

Finally, the decision-making style is influenced by research on adapting existing safety decisions with trust while keeping the logic structured and explainable [2]. The implementation follows that direction by using explicit rules for following and crossroad behavior and by exporting decision information that can be inspected after simulation runs.

1.4 Trust and Reputation

Trust is used in this thesis as an operational estimate of another vehicle's reliability in traffic behavior. It does not describe moral value and it is not a global certificate of safety. It expresses how much one vehicle currently expects another vehicle to behave predictably in a traffic situation. Vehicles cannot always be separated into safe and unsafe groups. A vehicle can behave consistently for some time and still become less predictable because of congestion, route changes, local conflict, or a sudden maneuver.

Trust matters for decisions because it helps an autonomous vehicle handle uncertainty. If nearby behavior appears reliable, the vehicle can avoid needless conservatism and keep motion smoother. If behavior appears uncertain or suspicious, the vehicle can react more carefully. In this way, trust does not replace basic safety rules. It changes how cautious the vehicle is inside a safety envelope that still has to avoid collisions and illegal movement [2].

In the implemented model, trust can come from direct observation or from reputation. Direct trust is based on what an observer vehicle has experienced itself, for example whether another vehicle behaved consistently with the expected immediate behavior. Reputation is shared information about a third vehicle received from another nearby vehicle. The distinction matters because direct experience and reported experience do not have the same strength.

Reputation helps when direct experience is still limited, but it cannot be accepted blindly. A recommender can be wrong, outdated, or unreliable. For this reason, recommendations are weighted by trust in the recommender [1]. A recommendation from a highly trusted vehicle can influence the observer more strongly, while a recommendation from a low trust vehicle has less effect.

Separating direct trust and reputation keeps the final trust value traceable. A value can be traced back to initial trust, direct observation, and weighted recommendations. During evaluation, the exported trust breakdown can show whether a decision was mainly caused by observed behavior, shared reputation, or the initial trust value.

1.5 Dynamic and Observer-relative Trust

Trust in this thesis is dynamic. It can increase or decrease during a simulation as new evidence becomes available. An initial trust value can still be used, for example when it is derived from a vehicle type, but it should not represent the vehicle for the whole run. If a vehicle repeatedly behaves as expected, trust in that vehicle can grow. If its behavior differs from what was expected, trust can decrease. The main difference from a static trust assignment is that the trust value is not fixed before the simulation starts.

Trust is also observer-relative. A vehicle does not have one universal trust score that is visible to every other vehicle in the network. Trust is stored as a relation between an observer and a target. Two observers can hold different trust values for the same target because they have seen different behavior, received different recommendations, or interacted with the target for different lengths of time.

This makes trust assessment decentralized. There is no central authority that assigns a final trust value to each vehicle. Instead, every

vehicle builds and updates its own view from its own observations and from recommendations received during local encounters.

Observer-relative trust matches the local nature of traffic knowledge. A target vehicle may be familiar to one observer and almost unknown to another. A global trust value would hide this difference and could make the system overconfident. The observer-relative model keeps uncertainty visible. Each vehicle adapts according to what it knows, not according to a single shared label.

Together, dynamic and observer-relative trust support gradual local adaptation. A vehicle does not need to wait for a severe event before changing its behavior. Smaller differences between expected and observed behavior can influence trust over time. The evaluation can then show how these trust changes affected later driving decisions.

1.6 Local Knowledge Exchange

Local knowledge exchange describes how reputation is shared between vehicles. Vehicles do not continuously share all trust values with all other vehicles. A recommendation can be exchanged only when two vehicles are close enough to be considered in contact during the simulation. Vehicles that rarely meet remain mostly dependent on direct experience.

The communication model stops at the trust layer. Real Cooperative ITS systems may use standardized messages [8], but the simulation does not model message encoding, packet loss, security certificates, or roadside infrastructure. The only rule is that trust knowledge can be shared after a nearby encounter.

Restricting exchange to local encounters keeps the model closer to traffic interaction. If reputation were broadcast globally, every vehicle would quickly receive similar information and the observer-relative character of trust would become weaker. Local exchange preserves differences between vehicle perspectives. One vehicle may learn about a target through repeated contact or recommendation, while another vehicle may still know almost nothing about the same target.

Local exchange also limits the influence of unreliable information. A recommendation can affect another vehicle only after a local encounter, and its effect is still weighted by trust in the recommender.

Older direct observations are also weakened when vehicles no longer meet, and recommendations depend on the observer's current trust and confidence in the recommender. As a result, shared recommendations are not treated as global or permanent truth.

The traffic scenario affects how local exchange develops. High traffic or intersection heavy scenarios create more repeated encounters, so trust and reputation can develop faster. Sparse scenarios contain fewer opportunities for repeated contact, so vehicles have less shared knowledge to use. The traffic network affects travel time, smoothness, and how trust knowledge spreads between vehicles.

1.7 Digital Twin

A digital twin is a runtime representation of a real or simulated entity. In this thesis, the term is used in a narrow way. The digital twin does not try to copy a complete vehicle controller or predict the full future route. It stores only the current behavior information needed to judge whether a vehicle acted as expected in the current traffic context.

The digital twin idea is useful because trust should be based on behavior, not only on labels. Related work on autonomous drones uses digital twins to represent declared behavior and compare it with real-time observations [7]. Here, the idea is reduced to traffic evidence. The observed vehicle is represented by a lightweight twin, the local scenario is created from SUMO state, and expected behavior is compared with what the vehicle actually did in the next steps.

This makes the trust update more explainable than a purely black-box score. The implementation can report that trust changed because the target differed from expected lane, speed, acceleration, or intersection behavior. Chapter 3 describes the concrete implementation.

1.8 Decision Rules

Trust affects driving only when it influences a decision. In this model, trust values are connected to explicit decision rules. These rules choose how cautious a vehicle should be in selected traffic situations, such as following another vehicle or approaching a crossroad. The rules are intentionally simple.

With explicit rules, the evaluation can connect a change in following distance or crossroad caution to a named policy branch. The exported decision information gives more context for interpreting the run.

Figure 1.1 shows the intended style of the decision logic. Only the general idea is taken from the figure, not the exact decision tree. The implemented rules are described in the implementation chapter, where the exact inputs, thresholds, and exported statistics are explained in detail.

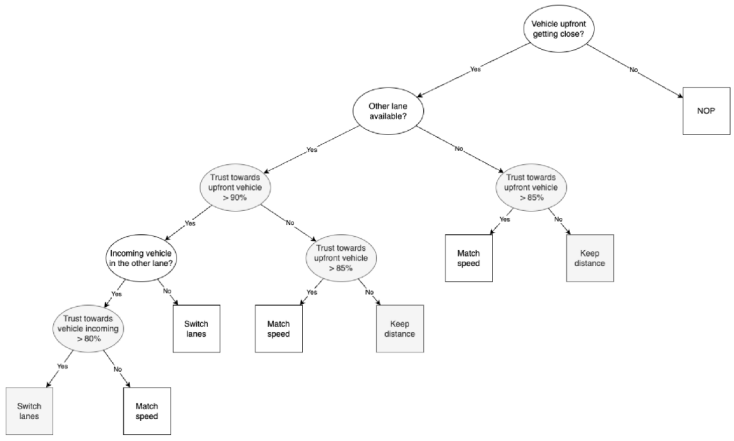


Fig. 4. Adjusted Decision Tree with trust-based adaptation

Figure 1.1: Structured illustration of trust-based adaptation [2].

2 Analysis and Planning

2.1 SUMO

SUMO was selected as the main simulation environment because it is an open source microscopic and continuous traffic simulator designed for road traffic experiments [9]. In a microscopic simulator, vehicles are represented individually rather than only as an aggregate flow. That matters here because trust is formed between specific vehicles. The simulation must keep enough information about each vehicle's position, lane, speed, route, and surrounding traffic context.

SUMO also provides the practical features needed for this thesis. Road networks and traffic demand can be defined in configuration files, simulations can be run repeatedly under scripted control, and the same scenarios can be executed with different vehicle behavior. This makes it possible to use the same traffic conditions for the adaptive and trust-driven variants. In this work, the relevant changes in vehicle behavior include following distance, crossroad caution, and local interaction between vehicles that build trust over time.

SUMO also keeps continuity with previous work in this research line. It was chosen by Richard Plesník [5] and later reused by Radovan Hančuľák [6]. The present thesis compares the proposed method with the existing adaptive variant, so using the same simulation environment keeps the comparison fair. The main experimental difference remains in the trust and decision logic rather than in the simulator or runtime interface.

Scenario Files

The simulation inputs are split across several SUMO artifacts instead of being embedded directly in the Python trust code. The network file stores the road layout, including lanes, junctions, speed limits, right-of-way rules, traffic light logic, and lane connections [10]. Route files store vehicle types, vehicles, routes, and departure definitions [11]. A SUMO configuration file then selects the input files for a concrete run.

Table 2.1: Main SUMO-related artifacts used by the simulation workflow.

File type	Example	Role in this thesis
Network	`.net.xml`	road layout, lanes, junctions, speed limits
Routes	`.rou.xml`	vehicle types, trust priors, traffic demand
Config	`.sumocfg`	scenario entry point used by SUMO
Output	`.txt`, `.csv`, `.png`	traffic, trust, and smoothness results

The table is included only to clarify which SUMO artifacts are part of the simulation setup. The trust-driven behavior itself is implemented in the Python control loop described in the following sections.

Vehicle Behavior and Randomness

SUMO vehicle type definitions contain physical values and following model parameters [11]. The default following behavior is based on the Krauß model [12, 13]. The implementation in this thesis does not replace SUMO's following model. Instead, it changes selected parameters and speed decisions through TraCI. The most important parameter is τ, which controls the desired time gap to the leader.

SUMO can also include stochastic behavior. Its random number generator is initialized with a seed, and repeated runs with the same inputs and same seed are expected to be reproducible in normal cases [14]. This is why the evaluation uses repeated runs and matching seed sequences for the compared methods. The purpose is to compare patterns across runs rather than to interpret a single random run as conclusive evidence.

2.2 TraCI

TraCI is used because the trust model has to react while the simulation is running. It provides online access to SUMO, so an external program can retrieve simulation values and change the behavior of simulated objects during execution [15]. The model needs this online access

because trust is built from current encounters, direct observations, exchanged recommendations, and local traffic context.

In this work, TraCI supplies the information needed by the trust pipeline. The simulation script can read which vehicles are active, inspect their speed, acceleration, lane, road, and position, detect leaders, and query lane speed limits. These values are used to identify nearby vehicle pairs, build evaluation context, and update observer-relative trust values during the run.

TraCI is also the path from trust assessment back to vehicle behavior. After a trust value is computed, the vehicle wrapper can apply the selected policy by changing SUMO parameters, such as the following parameter τ, or by reducing speed when the crossroad rule selects a cautious response. Observation, trust update, and behavior adaptation happen in the same simulation loop.

2.3 Chosen Metrics

The evaluation uses metrics that reflect the main goal of the thesis, improving traffic behavior without making the vehicles less cautious across the board. For that reason, the metrics are divided into three groups. The first group describes traffic flow, the second group describes smoothness and stability, and the third group contains trust and decision outputs.

The traffic flow metrics show whether the simulated road network processes vehicles efficiently. The smoothness and stability metrics show whether the method behaves steadily rather than gaining performance through abrupt local actions. The trust and decision outputs connect the measured traffic results with the internal trust behavior of the system.

2.3.1 Traffic Flow Metrics

Traffic flow is measured by total duration, cars per minute, and average travel time. Total duration describes how long the scenario demand takes to clear. Cars per minute approximates network throughput. Average travel time captures efficiency from the perspective of individual vehicles.

These metrics describe different views of the same result. Duration describes the whole scenario, cars per minute describes throughput, and travel time describes the experience of individual vehicles. When all three move in the same direction, the result is more credible. If only one improves while the others remain unchanged or worsen, the explanation has to be treated more cautiously.

2.3.2 Smoothness and Stability Metrics

Traffic flow alone is not enough for evaluating trust-driven adaptive safety. The method should also keep vehicle movement smooth. For this reason, the trust-driven simulation records average τ, average acceleration variance, average crossroad speed factor, and cautious slowdown counts. These metrics indicate whether traffic quality comes from stable behavior rather than abrupt changes.

Average τ captures how conservative the following policy became in practice. Acceleration variance approximates longitudinal smoothness. The average crossroad speed factor indicates whether the system often preserved intersection flow or often entered a cautious mode. The cautious slowdown count gives a more explicit signal of how often distrust became protective action.

2.3.3 Trust and Decision Outputs

The last group of outputs explains the traffic results. The simulation records nearby meetings, shared recommendations, direct evaluations, average visible trust, trust summaries, and trust breakdown summaries. These values show whether vehicles actually met, exchanged knowledge, and formed trust from initial values, direct observation, and reputation.

The simulation also exports decision rule counts and policy snapshots. Decision counts show which trust-driven rules were used during a run. Policy snapshots record the parameter values used for that run. These outputs do not replace traffic flow or smoothness metrics, but they add context to the result.

2.4 Simulation Scenarios

The main evaluation scenario is `simulation-complex`. The trust-driven method needs a scenario where vehicles meet repeatedly, follow one another, approach shared junctions, and have opportunities to exchange local trust knowledge. A simple free-flow route would not exercise enough of the model. The complex scenario provides more varied local interactions, so it is the primary setting for evaluating whether dynamic observer-relative trust changes traffic behavior.

The `simulation-complex` network contains several connected road segments and priority junctions. Vehicles enter from different routes and can meet around shared parts of the network. The scenario creates conditions for observing following behavior, crossroad caution, recommendation exchange, and the comparison between expected and observed behavior.

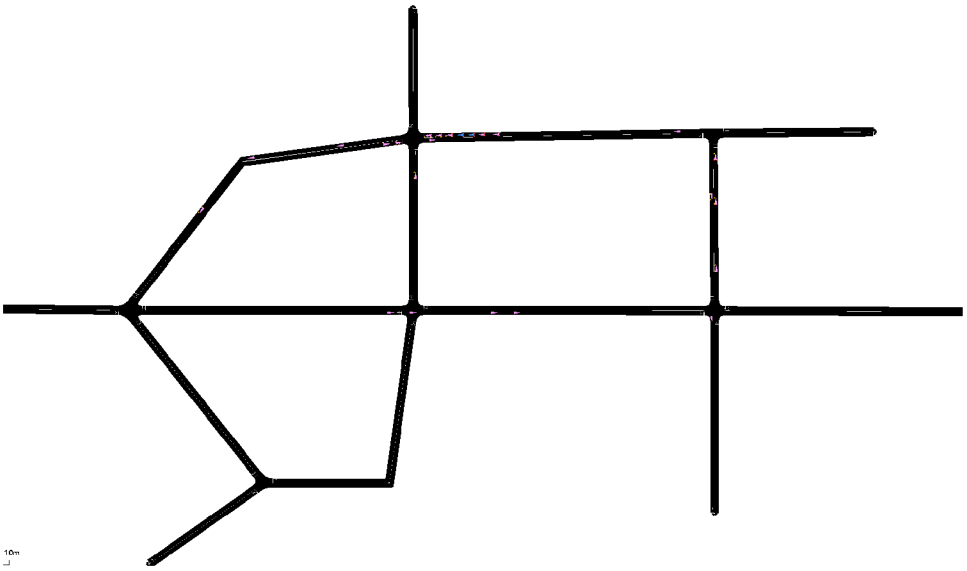


Figure 2.1: SUMO representation of the `simulation-complex` road network used as the primary interaction-rich scenario.

The evaluation also uses `simulation_highway_2`. Reusing this scenario keeps the road layout, traffic demand, and simulation environment aligned with the earlier adaptive variant [6]. The scenario gives

a familiar setting for checking whether the new trust-driven method improves on the older adaptive method.

`simulation_highway_2` contains a linear road where three lanes narrow to two and later widen again. This creates sustained following behavior, denser traffic, and a bottleneck where local decisions can affect throughput and travel time. It stresses a different part of the method than `simulation-complex`, where repeated local encounters and crossroad contexts matter more.

The standalone `simulation_crossroad` scenario was not selected as a main evaluation scenario. It can check isolated crossroad behavior, but it is narrower than the main claims of the thesis. It gives fewer opportunities for repeated trust exchange and longer traffic flow effects than the complex and highway scenarios.

2.5 Compared Trust Aware Methods

The comparison focuses on two trust aware variants, `adaptive` and `dynamic_trust`. Both variants use trust to change vehicle behavior, so they are close enough to compare directly. The main difference is how trust is represented, updated, and used during the simulation.

The `adaptive` variant represents the earlier trust safety method. It uses trust values assigned before the simulation and applies them to selected driving behavior, mainly following distance and crossroad caution. It is a close predecessor because it already uses trust to influence traffic behavior, but it does not model trust as a changing relation between individual vehicles.

The `dynamic_trust` variant is the method developed in this thesis. It stores trust from the perspective of each observer vehicle, updates trust during the run, exchanges recommendations only through nearby encounters, and weights shared knowledge by trust in the recommender. The same general driving situations are affected, but the trust values behind the decisions are dynamic and local.

Comparing these variants shows whether the dynamic observer-relative model improves on the earlier static trust approach. The evaluation checks whether traffic results improve and uses exported trust and decision outputs to explain how the trust-driven behavior was produced during the simulation.

The evaluation uses repeated runs rather than individual examples, because SUMO simulations can vary between runs. The scenarios, metrics, and compared variants are chosen so that the final evaluation can compare aggregated behavior across the same simulation settings.

3 Implementation

3.1 Codebase Overview

The implementation is built inside the existing simulation project rather than as a separate prototype.

The main entry point is `simulations/scripts/dynamic_trust.py`. The script starts the SUMO simulation through TraCI, creates the trust manager and metrics manager, and wraps active smart vehicles with the behavior used by the trust-driven method.

Simulation scripts and scenarios

The simulation scripts select which vehicle behavior is used in a run. The main script for this thesis is the trust-driven variant, and the earlier adaptive variant is kept for comparison. The relevant files are listed below.

- `simulations/scripts/dynamic_trust.py`
- `simulations/scripts/adaptive.py`
- `simulations/configs/`

The scenario folders contain the SUMO network, route, settings, and configuration files needed to run each traffic situation.

Shared simulation support

Shared support code is placed under `core/`. It contains the common network parsing, metric collection, and vehicle abstractions used by the simulation scripts. The most relevant parts are listed below.

- `core/net_parser.py`
- `core/metrics/`
- `core/cars/`

The trust-driven vehicle wrapper connects the current trust values to concrete vehicle behavior, mainly following distance and crossroad caution.

Trust-driven modules

Most of the new method is implemented in the trust implementation modules. The main files are listed below.

- `trust_manager.py`
- `trust_store.py`
- `digital_twin_manager.py`
- `scenario_generator.py`
- `digital_twin_simulator.py`
- `comparator.py`
- `decision_rules.py`
- `policy.py`

Together, these modules coordinate trust updates, store observer-relative values, build local evaluation context, compare expected and observed behavior, and turn trust into driving adjustments.

The layout keeps the main responsibilities separate. The simulation script controls the run. The trust modules compute and store trust. The vehicle wrapper applies the current decisions to SUMO vehicles. The metrics modules record the outputs used later in the evaluation. The separation makes it clearer which parts changed compared with the earlier adaptive method.

3.2 Runtime Loop

The main script runs the simulation as a repeated control loop. The loop continues until SUMO finishes the configured simulation. During each simulation step, the script refreshes the active vehicle set from SUMO, initializes newly entered smart vehicles, records the state needed for later evaluation, delegates trust updates between vehicles to the trust manager, applies trust-driven safety behavior to the active smart vehicles, and records metrics for the current step. At the

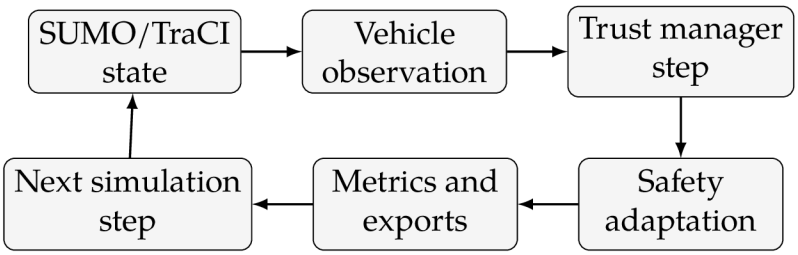


Figure 3.1: Runtime loop of the trust-driven simulation script. Each step observes vehicle state, updates trust, applies adaptation, and records metrics.

end of the run, it exports the traffic results, smoothness results, trust summaries, and decision outputs.

The following sections describe the two most important parts of this loop. The first is how trust is updated from local interaction. The second is how the resulting trust values change vehicle behavior.

Both variants can be executed through the same scenario folders, metric export logic, and comparison scripts. The main experimental difference is the trust and decision logic, not a different simulator setup.

3.3 Core Components

The trust pipeline follows the component structure described in related work [1]. The implementation uses the same main responsibilities, but adapts them to the SUMO and TraCI simulation setting.

Trust Manager

The Trust Manager coordinates the runtime trust process. It receives the current simulation state, finds relevant nearby vehicles, triggers knowledge exchange, starts direct evaluation, computes final trust values, and exports trust statistics. It is the central component that connects local traffic interaction with the rest of the trust pipeline.

Digital Twin Manager

The Digital Twin Manager creates a small representation of the target vehicle. The representation contains information used for local behavior comparison, such as current movement, lane context, and stability. It does not try to model the full internal state of the vehicle.

Scenario Generator

The Scenario Generator adds local traffic context around the target vehicle. It uses the current SUMO state to describe the relation between observer and target, nearby leaders, gaps, speed limits, and intersection context. The later prediction step then has enough information to judge the target behavior in its current situation.

Digital Twin Simulator

The Digital Twin Simulator predicts expected immediate behavior from the digital twin and the generated scenario. The prediction is intentionally local. It is used to estimate what behavior would be reasonable in the current traffic context, not to plan a complete future route.

Comparator

The Comparator checks whether the target vehicle behaved close to the prediction. It starts from a good score and subtracts small penalties when the observed behavior differs too much. The comparison looks at the lane, speed, acceleration, and behavior near intersections. The resulting score is then used as direct evidence for the observer's trust in the target vehicle.

The components are kept separate so that each part has a clear role. One part stores and updates trust. Another builds the local traffic situation. Another predicts expected behavior, and another compares the prediction with what happened in SUMO. This keeps the digital twin architecture easy to follow.

During execution, the components form a repeated local assessment loop. When two vehicles become relevant to each other, the trust manager treats one as the observer and the other as the target. The

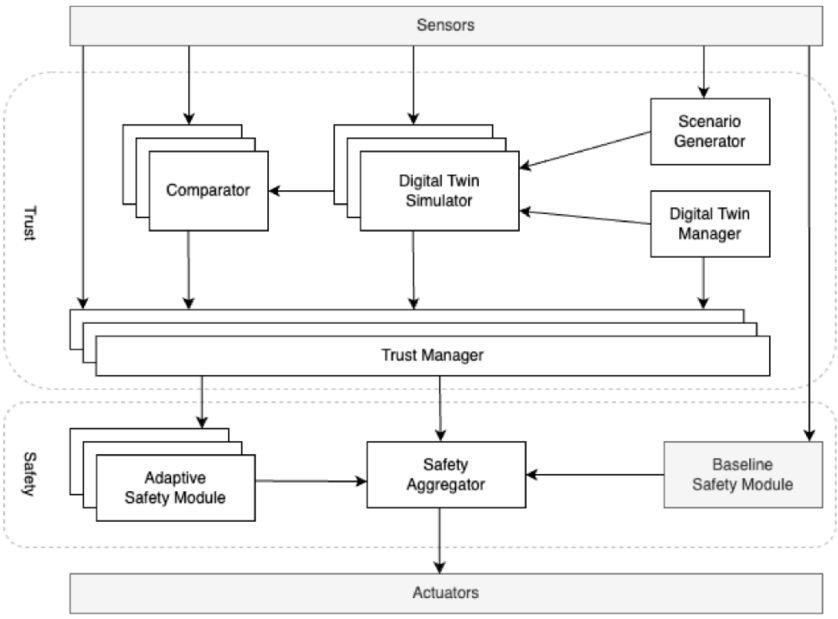


Figure 3.2: High-level trust architecture [1].

target is represented by a digital twin. A local scenario is built from the current SUMO state. The digital twin simulator predicts plausible target behavior in that context, and the comparator checks whether the observed behavior is consistent with the prediction. The loop is local. It does not solve a global planning problem. It produces evidence for trust updates.

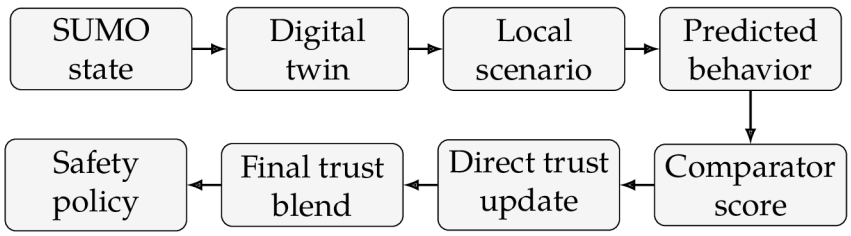


Figure 3.3: Trust update flow used during one simulation step.

Because these components are separate, the digital twin, scenario generator, or comparator could be changed later while the rest of the design remains intact.

3.4 Trust Storage and Update Logic

Stored Trust Data

The trust state is kept in a dedicated store. It records initial trust values, observer-relative direct trust values, contact step counts, shared opinions, encounter counts, and the latest breakdown of each final trust value. The store keeps these parts separate so they can be exported and interpreted later.

Initial trust enters through the vehicle type suffix, for example `smart_car_30` or `smart_car_65`. Direct trust is updated from the consistency of vehicle behavior during simulation. Reputation comes from shared opinions received from other nearby vehicles. The final value can come from an initial value, direct evidence, reputation, or a blend of direct evidence and reputation. This follows IoV trust work that separates direct trust from indirect trust based on recommendations from nearby vehicles [16]. The model here keeps this idea local to the observer and target vehicle. It does not use the source paper's roadside unit aggregation or global trust score.

Direct Confidence

Direct confidence grows with contact duration. For observer o and target t, the raw confidence is

$$c_{o,t}^{raw} = \min\left(\frac{\text{contact_steps}_{o,t}}{C}, 1\right), \tag{3.1}$$

where C is the configured contact confidence limit. In the current policy, $C = 30$. The value $c_{o,t}^{raw}$ only reflects contact duration. The value $c_{o,t}$ used later is the adjusted confidence. It also includes freshness. After a grace period without contact, it is reduced by a recency factor, so old contact does not remain fully authoritative.

$$c_{o,t} = c_{o,t}^{raw} \cdot f_{o,t}^{recency} \tag{3.2}$$

The freshness factor $f_{o,t}^{recency}$ is equal to 1 while the vehicles have met recently. If they do not meet for longer, the factor gradually decreases down to a configured minimum value. This is a simpler version of the

time decay idea used in related IoV trust work, where older interaction evidence receives less weight [16].

Direct Trust Update

New direct evidence is incorporated through a smoothing coefficient that depends on confidence.

$$\alpha_{o,t} = A_{min} + A_{range} c_{o,t} \tag{3.3}$$

The coefficient $\alpha_{o,t}$ controls how strongly a new observation can change direct trust. It depends on $c_{o,t}$, so it also depends on how long the observer and target have been in contact and how fresh that contact is. With low confidence, $\alpha_{o,t}$ is smaller. With high confidence, it is larger. The constant A_{min} gives every new direct observation at least some influence, while A_{range} controls how much this influence can grow when the contact history is stronger. In the current policy, $A_{min} = 0.15$ and $A_{range} = 0.35$. For example, if $c_{o,t} = 0$, then $\alpha_{o,t} = 0.15$. If $c_{o,t} = 1$, then $\alpha_{o,t} = 0.50$. A new observation can update direct trust weakly when confidence is low and more strongly when confidence is high.

$$D_{o,t}^{new} = (1 - \alpha_{o,t}) D_{o,t}^{old} + \alpha_{o,t} S_{obs} \tag{3.4}$$

This formula then uses $\alpha_{o,t}$ to update direct trust. The value $D_{o,t}^{old}$ is the previous direct trust that observer o had for target t. The value S_{obs} is the observation score returned by the comparator for the current behavior of target t. The result $D_{o,t}^{new}$ is the updated direct trust after the new observation is included.

The first part, $(1 - \alpha_{o,t}) D_{o,t}^{old}$, keeps a portion of the previous direct trust. The second part, $\alpha_{o,t} S_{obs}$, adds the new observation score. A smaller $\alpha_{o,t}$ keeps more of the old direct trust. A larger $\alpha_{o,t}$ gives more influence to the new observation score.

For example, suppose observer o currently has direct trust $D_{o,t}^{old} = 0.70$ in target t. The comparator returns $S_{obs} = 0.90$, and the current update coefficient is $\alpha_{o,t} = 0.30$. The updated direct trust is then

$$D_{o,t}^{new} = (1 - 0.30) \cdot 0.70 + 0.30 \cdot 0.90 = 0.76. \tag{3.5}$$

The value moves from 0.70 toward the new observation score, but it does not jump to 0.90 immediately. Direct trust is also pulled back

toward the target initial value when the observer and target have not met for a longer time. Old direct experience remains available without dominating indefinitely.

Reputation

Reputation $R_{o,t}$ represents what observer o knows about target t through other vehicles. It is used when observer o has received shared opinions about target t from nearby vehicles. These vehicles are called recommenders. Siddiqui et al. compute indirect trust from neighboring vehicles' direct trust values and weight each neighbor by a confidence factor [16]. The formula here uses the same idea in a simpler form. Each shared opinion comes from a recommender's direct trust in the target, and the observer gives that opinion more or less weight according to its trust and confidence in the recommender.

The set $Rec(o,t)$ contains all recommenders that shared an opinion about target t with observer o. For example, if vehicles b and c both shared their opinion about vehicle t with observer o, then $Rec(o,t) = \{b,c\}$. The sum in the formula goes through this set one recommender at a time.

$$R_{o,t} = \frac{\sum_{r \in Rec(o,t)} q_{o,r}\, opinion_{r,t}}{\sum_{r \in Rec(o,t)} q_{o,r}}. \tag{3.6}$$

The value $opinion_{r,t}$ is recommender r's shared opinion about target t. In the implementation, this opinion is based on recommender r's direct trust in target t. A vehicle does not share a global truth about another vehicle. It shares its own direct experience.

The value $q_{o,r}$ is the weight assigned to recommender r by observer o. It describes how much observer o should rely on recommender r's opinion. This weight is higher when observer o has stronger trust and confidence in recommender r. If observer o has little direct experience with recommender r, the recommender can still contribute, but its opinion receives less weight.

The numerator adds all shared opinions after multiplying each opinion by its recommender weight. The denominator adds the recommender weights themselves. Dividing by the denominator makes the result a weighted average rather than a simple sum. Adding more

recommenders does not automatically make the reputation larger. It only gives the observer more weighted information.

For example, suppose observer o receives two opinions about target t. Recommender b reports $opinion_{b,t} = 0.8$ and has weight $q_{o,b} = 0.9$. Recommender c reports $opinion_{c,t} = 0.4$ and has weight $q_{o,c} = 0.3$. The reputation is then

$$R_{o,t} = \frac{0.9 \cdot 0.8 + 0.3 \cdot 0.4}{0.9 + 0.3} = 0.7. \tag{3.7}$$

The result is closer to recommender b's opinion because observer o gives more weight to b than to c.

Final Trust Value

When direct trust and reputation are both available, their final weights depend on direct confidence. Related IoV trust work also combines direct and indirect trust with a weighted sum [16]. In that paper, the weight depends on interaction frequency. In this thesis, the weight depends on direct confidence. The exact constants used here are policy parameters of this implementation.

$$w_d = W_{d,min} + W_{d,range} c_{o,t}, \tag{3.8}$$

$$w_r = 1 - w_d. \tag{3.9}$$

w_d is the weight of direct trust, and w_r is the weight of reputation. The constant $W_{d,min}$ gives direct trust a starting weight when both direct trust and reputation are available. $W_{d,range}$ controls how much this weight can grow with direct confidence. In the current policy, $W_{d,min} = 0.45$ and $W_{d,range} = 0.35$. As direct confidence grows, direct trust receives a larger part of the final value.

For example, if $c_{o,t} = 0$, then $w_d = 0.45$ and $w_r = 0.55$. Reputation has slightly more influence because direct confidence is still low. If $c_{o,t} = 1$, then $w_d = 0.80$ and $w_r = 0.20$. Direct trust then has most of the influence, but reputation still remains part of the final value.

The final observer-relative trust value is then

$$T_{o,t} = w_d D_{o,t} + w_r R_{o,t}. \tag{3.10}$$

Here $T_{o,t}$ is the final trust of observer o in target t. The value $D_{o,t}$ is direct trust, and $R_{o,t}$ is reputation. The formula combines both values according to their weights. For example, suppose $D_{o,t} = 0.80$, $R_{o,t} = 0.50$, $w_d = 0.80$, and $w_r = 0.20$. The final trust is then

$$T_{o,t} = 0.80 \cdot 0.80 + 0.20 \cdot 0.50 = 0.74. \tag{3.11}$$

The result is closer to direct trust because the direct confidence is high and because w_d is larger.

Reputation can help early in an encounter, but direct evidence can dominate after repeated interaction.

3.5 Local Interaction and Knowledge Exchange

The Trust Manager builds the runtime logic on top of the trust store. It removes inactive vehicles, collects the current simulation state, and identifies nearby pairs. Pair detection uses spatial buckets before the exact meeting radius check is made. The implementation follows the general spatial hashing idea used to optimize large scale simulations [17]. It avoids checking every possible pair when only nearby vehicles can exchange knowledge.

The earlier version checked the distance between every pair of active vehicles. That version became too slow as the number of vehicles increased, because most vehicles were far away from each other and could not interact anyway. The current version first places each vehicle into a square bucket according to its position. In the current policy, the bucket size is 40 meters. When searching for possible nearby pairs, the trust manager only compares vehicles from the same bucket and from the neighboring buckets. The exact meeting radius check is still used afterwards, so the buckets only reduce the number of candidate pairs. They do not decide by themselves whether two vehicles have met.

Figure 3.5 shows the corresponding implementation idea. The bucket position is computed from the vehicle coordinates, nearby buckets are used to collect candidates, and the final nearby decision is still made by the real distance check.

A nearby pair is not always evaluated directly. The implementation evaluates a pair only when the vehicles are likely to affect each other

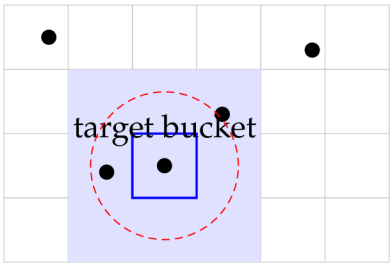


Figure 3.4: Spatial bucketing.

```
# Assign vehicles to spatial buckets.
bucket_key = (
  int(x_pos // self.SPATIAL_CELL_SIZE),
  int(y_pos // self.SPATIAL_CELL_SIZE),
)
buckets[bucket_key].append(car_id)

# Compare only the current bucket and neighbouring buckets.
for dx in (-1, 0, 1):
  for dy in (-1, 0, 1):
      candidate_ids.extend(
          buckets.get((bucket_x + dx, bucket_y + dy), [])
      )

# The real distance is checked afterwards.
if self._cars_are_nearby(first_id, second_id, states):
  pairs.append((first_id, second_id))
```

Figure 3.5: Spatial bucketing implementation.

in traffic. This is the case when both vehicles are on the same road or same crossroad.

A typical trust interaction starts when two vehicles enter the configured nearby radius. The encounter is treated as a local chance for observation and limited knowledge exchange. The trust manager records the meeting, updates contact step counts, and exchanges known direct opinions when the sharing interval allows it.

Figure 3.6 shows the exchange step. The provider first selects vehicles for which it has direct trust information. It skips itself and the receiving vehicle. For each remaining target, the currently effective direct trust value is stored as a shared opinion for the receiver.

```
known_targets = set(self.store.direct_trust[provider_id].keys())
for target_id in known_targets:
    if target_id in {provider_id, receiver_id}:
        continue
    opinion = self._get_effective_direct(
        provider_id,
        target_id,
        self.store.get_prior(target_id),
    )
    if opinion is not None:
        self.store.store_shared_opinion(receiver_id, provider_id, target_id, opinion)
        self.total_recommendations_shared += 1
```

Figure 3.6: Local knowledge exchange.

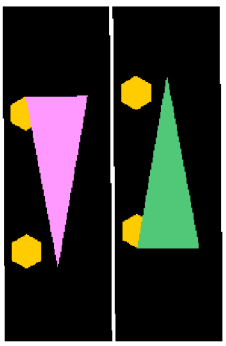


Figure 3.7: Example of nearby vehicles meeting in the simulation, triggering local trust exchange and direct trust evaluation.

Knowledge exchange remains local to nearby vehicles. Vehicles do not publish their trust values to a central store, and they do not

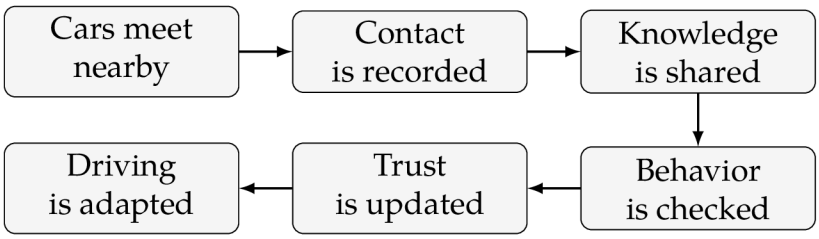


Figure 3.8: Nearby vehicle encounter.

continuously receive opinions about all vehicles in the network. Reputation spreads gradually through physical encounters. The trust model remains decentralized and more similar to a real traffic scenario.

3.6 Digital Twin Based Evaluation

Direct trust is updated through a local behavior comparison between expected and observed behavior. For the observed target, the Digital Twin Manager builds a digital twin from the current SUMO state. The twin records the target vehicle's current lane, declared speed, and stability estimate.

The Scenario Generator then adds local context. It records the distance between observer and target, the target's crossroad context, the current leader, the gap to that leader, the leader's speed, the target's speed, and the local speed limit. The Digital Twin Simulator uses these inputs to predict expected lane, expected speed, speed tolerance, maximum expected acceleration magnitude, and whether intersection caution should apply. The design follows the general idea that a digital twin can describe expected behavior and that observed behavior can be checked against it during operation [7].

Figure 3.9 shows the evaluation step in code. The observed target is used to build a digital twin, a local scenario is generated, and the prediction is compared with the actual state. The resulting observation score is then used to update direct trust.

The Comparator starts from an ideal score and subtracts explicit penalties.

```
target_car = cars[target_id]
twin = self.digital_twin_manager.build_twin(target_car, states[target_id])
scenario = self.scenario_generator.generate(
    observer_id, target_id, states, crossroad_cache, leader_cache
)
prediction = self.digital_twin_simulator.predict(twin, scenario)
actual = states[target_id]
observation_score, _ = self.comparator.compare(prediction, actual, scenario)
self.total_evaluations += 1
self._blend_direct_trust(observer_id, target_id, observation_score)
```

Figure 3.9: Digital twin based evaluation.

$$S_{obs} = \max\left(0, \min\left(1, 1 - p_{lane} - p_{speed} - p_{accel} - p_{intersection}\right)\right). \tag{3.12}$$

The lane part checks whether the vehicle stayed in the expected lane when it was not at an intersection. The speed part checks how far the observed speed was from the expected speed. The acceleration part catches unusually strong acceleration or braking. The intersection part is used when the vehicle moves too quickly in a situation where cautious behavior was expected.

Queueing at an intersection is treated separately. In an earlier version, vehicles that were standing in a queue could receive too large a penalty because their low speed and braking did not match normal free movement. Normal waiting could look like incorrect behavior. The current version is less strict when a vehicle is slow or braking because it is waiting in traffic. Normal queueing behavior now does not lower trust unnecessarily.

3.7 Safety Adaptation

Trust affects two parts of vehicle behavior related to safety. The first is following distance, implemented through the SUMO parameter τ. The second is crossroad speed adaptation, where the vehicle becomes more or less cautious around other cars near the same intersection.

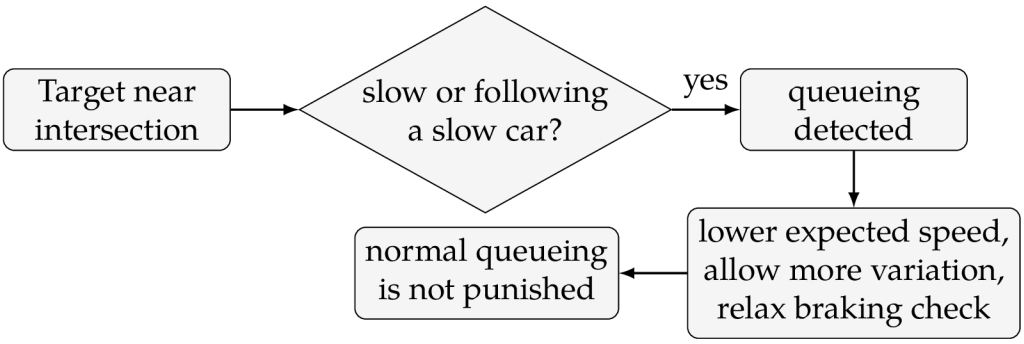


Figure 3.10: Intersection queue detection and evaluation.

3.7.1 Following Distance

When following another vehicle, the smart vehicle first asks TraCI for its current leader. It then asks the Trust Manager how much it trusts that leader. The trust value is used to choose the time headway parameter τ, which controls how much time gap SUMO keeps behind the leader. Higher trust allows a shorter gap. Lower trust keeps the vehicle more cautious.

The selected rule is also recorded for evaluation as `high_trust_leader`, `medium_trust_leader`, or `low_trust_leader`.

The mapping from trust to τ is split into three trust ranges. Each range has its own linear rule and lower and upper bound.

$$\tau(T) = \begin{cases} \max(0.6, \min(2.2, 2.15 - 1.70T)), & T \geq 0.80, \\ \max(0.7, \min(2.3, 2.28 - 1.50T)), & 0.52 \leq T < 0.80, \\ \max(1.0, \min(2.4, 2.45 - 1.10T)), & T < 0.52. \end{cases} \tag{3.13}$$

The numbers are policy parameters chosen for this implementation. They are not general traffic constants. The bounds prevent τ from becoming too small or too large. Within each range, higher trust lowers τ, which lets the vehicle follow the leader more closely. Lower trust keeps τ higher and leaves more space.

A separate adjustment is used when the vehicle is following another vehicle on a highway. In this case, the vehicle may reduce τ slightly more, but only under strict conditions. The leader must have enough trust and direct confidence. Both vehicles must be moving fast

and at a similar speed. The gap must be neither too small nor too large. The lane must support higher speed, and neither vehicle may be close to an intersection. When all these conditions hold, the adjustment is

$$\tau_{highway} = \max(0.5, \tau - 0.14), \tag{3.14}$$

The extra reduction is intentionally small. It is meant for stable highway movement, not for intersections or uncertain local situations.

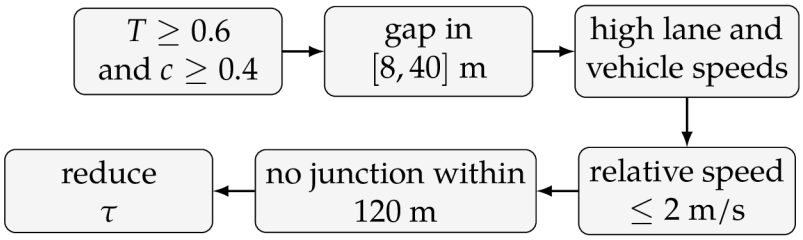


Figure 3.11: Highway following distance conditions.

3.7.2 Crossroad Behavior

At a crossroad, the vehicle looks at other relevant vehicles at the same intersection. For each of them, it retrieves the final observer-relative trust value and its trust breakdown. The breakdown matters because not all trust values should influence crossroad caution equally. A value based only on initial trust is weaker than a value supported by direct observation. Reputation is also treated carefully, especially when direct confidence is still low.

The basic distrust value is the opposite of trust.

$$d_{o,t} = 1 - T_{o,t}. \tag{3.15}$$

The implementation then adjusts this distrust value according to how the trust value was formed. Distrust based only on initial trust is reduced the most, because it is weak evidence. Distrust based only on reputation is also reduced, but less strongly. When direct trust and reputation are blended, the distrust is reduced if direct confidence is still low. Trust values supported by direct evidence are used without this reduction.

The adjusted distrust values are then averaged across the relevant vehicles at the crossroad. The vehicle also counts how many of them have sufficiently low trust. Together, these values select one of three named rules. `trusted_crossroad_flow`, `mixed_trust_crossroad`, or `low_trust_crossroad_cluster`. The selected rule determines the speed factor applied through TraCI.

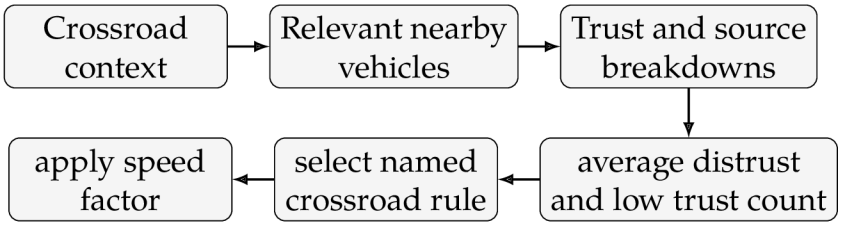


Figure 3.12: Crossroad speed adaptation.

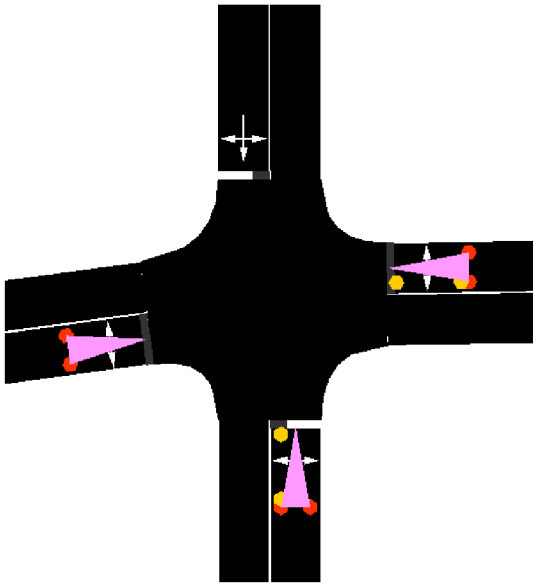


Figure 3.13: Example of vehicles approaching an intersection, where the crossroad trust policy decides whether flow is preserved or caution is increased.

3.8 Policy Parameters and Exported Outputs

The main tunable constants are kept in the policy module. The constants cover the meeting radius, contact confidence limit, evaluation interval, knowledge sharing interval, spatial cell size, trust thresholds, τ parameters, crossroad thresholds, distrust weights, and highway conditions. The same module also stores parameters for confidence freshness and stale direct trust.

At the end of each run, the active policy values are exported to `policy_snapshot.txt` and `policy_snapshot.csv`. These files identify which parameter values produced a given result folder.

The implementation also uses color as a visualization aid during GUI runs. Vehicle color is based on average visible trust across observers, not on a single global trust value owned by the target vehicle. The display remains suitable for quick inspection while preserving the observer-relative trust model used by the actual decision logic.

Policy Snapshot

The files `policy_snapshot.txt` and `policy_snapshot.csv` record the policy values used in the run. A typical entry looks as follows.

```
contact_confidence_steps,30
```

The value means that direct confidence reaches its full raw value after 30 contact steps. The files show which settings were used for the run.

Trust Summary

The file `trust_summary.csv` records final trust values between observers and targets at the end of a run. Each row contains an observer vehicle, a target vehicle, the final trust value, contact steps, and encounter count.

```
veh001,veh002,0.3,107,1
```

The row means that vehicle `veh001` ended the run with trust value 0.3 toward `veh002`, after 107 contact steps and one encounter.

Trust Breakdown Summary

The file `trust_breakdown_summary.csv` explains how final trust values were formed. It stores contact steps, encounter count, the source of the value, the initial trust, direct trust, reputation, direct confidence, direct weight, reputation weight, and final trust. The following row shows a blended value.

```
veh001,veh002,107,1,blended,0.3,0.3,0.9854,1.0,0.8,0.2,0.4371
```

The row means that vehicle `veh001` stored trust in `veh002` after 107 contact steps and one encounter. The final value is blended from direct trust and reputation, with direct confidence 1.0, direct weight 0.8, and reputation weight 0.2. The file links a traffic result to the trust information behind it.

Decision Statistics

The files `decision_statistics.txt` and `decision_statistics.csv` record how often each following distance rule and crossroad rule was used.

```
distance,high_trust_leader,18727
```

The row means that the high trust leader rule was used 18,727 times during the run. These counts show which trust-driven decisions were active, not only whether the final traffic metrics changed.

Trust Statistics

The file `trust_statistics.txt` summarizes activity inside the trust manager. It records the number of trust manager steps, nearby meetings, shared recommendations, direct evaluations, and average visible trust. For local meetings, the file can contain the following value.

```
Nearby meetings: 44882
```

The value gives the number of local meetings recorded during the run.

Smoothness Statistics

The files `smoothness_statistics.txt` and `smoothness_statistics.csv` summarize behavior related to smoothness and caution. They record average τ, average acceleration variance, average crossroad speed factor, and cautious slowdown count.

Together, these exports connect the implementation with the evaluation. They show both the final traffic results and the underlying trust and decision activity behind those results.

4 Evaluation

4.1 Evaluation Workflow

One simulation run is not enough to compare the methods reliably. SUMO contains random variation, so one run can produce traffic that is less or more congested than usual. For this reason, the evaluation uses batches of repeated runs and compares their average values.

Final evaluation runs were produced as batches with fixed SUMO seeds. The seed sequence starts at 1000 and increases by one for each run. The compared methods use the same seeds. This makes the comparison more consistent. The helper script `scripts/runV2.sh` implements this workflow. It was extended during the thesis work so that each new batch archives previous active results, records the current git commit, captures SUMO warnings, and runs `calc.py` after the batch to aggregate the exported result folders into a comparison report. The comparison report stores the git commit hash of the code version that produced the run. This made it easier to compare results from different implementation versions during development.

During development, the optional `-parallel` parameter was used to speed up multi-run testing. This option uses GNU Parallel [18] to start the selected simulation scripts for the same seed at the same time. It only changes how the commands are launched. Each SUMO run still receives an explicit seed and writes its own result folder. The final results were run without this option.

4.2 Evaluation Scenarios

Complex Scenario

The complex scenario, stored as `simulation-complex`, uses a road layout with repeated crossroad situations and many opportunities for nearby trust exchange.

Complex Scenario With Own Routes

The `simulation-complex_own_routes` scenario keeps the same road layout as the complex scenario, but assigns routes to vehicles individually. It is used as a secondary check to see whether the result also holds with more varied vehicle routes. Individual routes can lead to shorter or less repeated meetings between the same vehicles, which gives the trust exchange a slightly different setting.

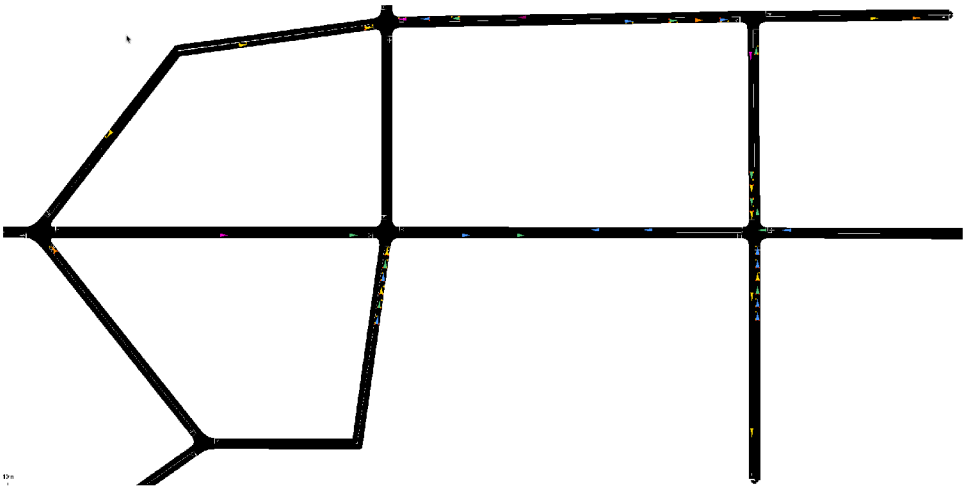


Figure 4.1: SUMO representation of the complex scenario with own routes.

Highway Scenario

The `simulation_highway_2` scenario emphasizes sustained leader following in high traffic.

4.3 Current Results

The result tables use the same repeated run setup. Each compared variant is run for 50 iterations under the same scenario settings. The tables report averaged values from `car_statistics.txt` files collected across the repeated runs. For cars per minute, higher values are better. For duration and average travel time, lower values are better.

Complex Scenario

Table 4.1 shows the results for the complex scenario. The trust-driven approach improves cars per minute by 20.58% compared with adaptive. It reduces duration by 16.96% and average travel time by 12.13%.

Table 4.1: Comparison across 50 runs on the complex scenario.

Metric	Adaptive	`dynamic_trust`	Improvement
Iterations	50	50	–
Duration (s)	178.71	148.40	-16.96%
Cars/min	16.12	19.44	+20.58%
Travel time (s)	126.81	111.43	-12.13%

Complex Scenario With Own Routes

Table 4.2 shows the results for the complex scenario with own routes. The results also improved in this scenario. Cars per minute increased by 9.78% compared with adaptive, while duration decreased by 8.91% and average travel time decreased by 7.80%.

Table 4.2: Comparison across 50 runs on `simulation-complex_own_routes`.

Metric	Adaptive	`dynamic_trust`	Improvement
Iterations	50	50	–
Duration (s)	149.34	136.06	-8.91%
Cars/min	19.32	21.20	+9.78%
Travel time (s)	76.54	70.57	-7.80%

Highway Scenario

Table 4.3 shows the results for the highway scenario. The strongest improvements are in this scenario. Compared with adaptive, the trust-driven approach improves cars per minute by 13.15%, reduces duration by 11.59%, and reduces average travel time by 22.30%.

Table 4.3: Comparison across 50 runs on `simulation_highway_2`.

Metric	Adaptive	`dynamic_trust`	Improvement
Iterations	50	50	–
Duration (s)	310.26	274.31	-11.59%
Cars/min	41.00	46.39	+13.15%
Travel time (s)	115.02	89.37	-22.30%

Scenarios Summarized

The results cover two different traffic situations. The highway scenario is most closely connected to the following distance rule described in Section 3.7.1. It contains many stable leader following situations, so improvements there are mainly evidence for the part of the method that adapts τ according to trust.

The complex scenarios exercise different parts of the implementation. Their repeated intersections relate to the crossroad behavior described in Section 3.7.2, while their local meetings relate to the knowledge exchange described in Section 3.5. The method is tested on both highway traffic and intersection traffic.

4.4 Trust and Decision Outputs

The traffic tables show the final effect of the compared methods. The exported trust and decision files are not a replacement for these traffic results. They are used for interpretation. Their role is to check whether the measured improvements match the trust-driven behavior that the method was designed to produce.

Decision Statistics

The decision counts support the interpretation that the highway result is connected to trusted leader following. The highway scenario should mainly benefit from following distance adaptation, and the exported rules match that expectation. Most recorded decisions were high trust following decisions, while medium and low trust following decisions still appeared. The method did not use only one rule. The

highway adjustment was used mostly when the leader had high trust, as intended.

Trust Statistics

The trust statistics show that the trust system was active during the evaluated runs. Vehicles repeatedly met, shared recommendations, and received direct evaluations. The traffic improvements were measured in runs where vehicles were actually forming and exchanging trust information.

Trust Summary

The trust summary is used mainly as a consistency check. It shows that trust is evaluated between an observer and a target, not as one global value attached to each vehicle. It also separates vehicle pairs that met from pairs that never interacted.

Trust Breakdown Summary

The trust breakdown is also an interpretation output, not a primary performance metric. It separates final trust values by source, including initial trust, direct observation, reputation, or a combination of sources. After separating out pairs that never met, the own routes run mostly produced blended trust among pairs that both met and formed direct evidence. This is consistent with how the implementation combines direct observation with shared recommendations in the final trust value.

Smoothness Statistics

The smoothness outputs help check whether the traffic gains came with reasonable vehicle behavior. In the highway results, no crossroad slowdowns were recorded, so the result mainly reflects following distance adaptation. That is expected because the highway scenario does not create the same kind of crossroad situations. In the complex scenarios, cautious slowdowns are recorded, which shows that the crossroad rule is active where intersection behavior matters. The own

routes result contains only a small number of cautious slowdowns, so the rule does not dominate the traffic behavior.

4.5 Malicious Actors

Malicious scenarios are used as an additional stress test. They are not the main evidence for the method, because the malicious behavior is intentionally simple. The purpose is to check how the evaluated methods behave when some vehicles do not follow normal cooperative behavior.

The malicious highway scenario inserts malicious vehicles into the highway traffic. Each malicious vehicle can perform one abrupt lane change. This maneuver fits the highway layout because the road has multiple lanes and a sudden lane change can force nearby vehicles to brake.

The comparison is prepared for two variants, `adaptive_malicious` and `dynamic_trust_malicious`. Both variants were run for 50 iterations with the same seed sequence.

Traffic Results

Table 4.4 shows the current traffic results for the malicious highway scenario. The dynamic malicious variant improves the main traffic metrics compared with `adaptive_malicious`. It lowers average duration and average travel time, while increasing cars per minute.

Table 4.4: Malicious highway traffic results.

Metric	Adaptive malicious	Dynamic malicious	Improvement
Iterations	50	50	–
Duration (s)	306.45	270.53	-11.72%
Cars/min	41.51	47.04	+13.33%
Travel time (s)	113.54	88.62	-21.94%

Warning Results

The warning output gives a more direct safety check. SUMO warnings were collected from the simulation output. Table 4.5 shows the warning counts from both 50 run batches. The dynamic malicious variant produced fewer warnings overall. It also did not produce collision warnings in this run. Emergency stop warnings increased from one to three, but this is a small part of the total warning output.

Table 4.5: Malicious highway warning results.

Metric	Adaptive malicious	Dynamic malicious	Improvement
Total warnings	92	53	-42.39%
Warnings per run	1.84	1.06	-42.39%
Median warnings per run	2.00	1.00	-50.00%
Minimum warnings per run	0	0	–
Maximum warnings per run	6	4	-33.33%
Emergency braking warnings	89	50	-43.82%
Emergency stop warnings	1	3	+200.00%
Collision warnings	2	0	-100.00%

The detailed warning split follows the same pattern as the total warning count. Most warnings are emergency braking events, and the dynamic variant clearly reduces them. The dynamic run also produced no collision warnings. Together with the lower total warning count, this supports the result that the dynamic variant handled the abrupt lane changes better in this scenario.

Dynamic Malicious Trust Comparison

The dynamic malicious variant also exports trust breakdowns. Among pairs where a normal observer actually met the target vehicle, normal targets received an average final trust of 0.72. Malicious targets received an average final trust of 0.41. This difference is shown in Figure 4.2.

This does not mean that every malicious vehicle was always identified as low trust. Not every vehicle was close enough to observe the

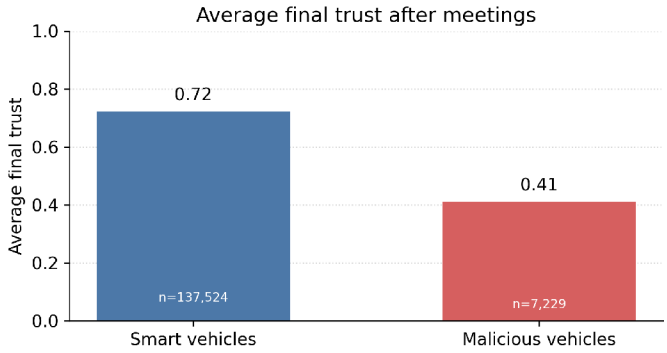


Figure 4.2: Average final trust assigned by normal vehicles after meetings in the malicious highway scenario. Here, n is the number of met observer-target pairs in each group.

malicious lane change when it happened. It does show that the trust update and recommendation parts of the method reacted differently to malicious targets than to normal targets.

5 Conclusion

The work presented a trust-driven adaptive safety approach for autonomous vehicle simulation in SUMO and TraCI. The motivation was to let simulated vehicles adapt selected safety decisions according to trust while still keeping the behavior clear enough to inspect.

The implementation follows a modular design described in related work [1]. It separates trust management, digital twin construction, scenario generation, prediction, and comparison. It also follows work on trust-based safety adaptation [2] by connecting trust to explicit safety decisions that can be inspected. These ideas are implemented through observer-relative trust computation, local knowledge exchange, direct and reputation trust blending, named decision rules, and trust and decision outputs.

The implemented system runs inside the existing SUMO and TraCI simulation project. In the validated scenarios, all reported traffic metrics improve compared with the adaptive variant while the design remains modular. The `simulation_highway_2` results show that the trust-driven approach improves cars per minute by 13.15%, reduces duration by 11.59%, and reduces average travel time by 22.30%. The complex scenario results show an improvement of 20.58% in cars per minute, a 16.96% reduction in duration, and a 12.13% reduction in average travel time.

The main limitation is runtime cost. Compared with adaptive, the `dynamic_trust` method performs more work because it tracks local meetings, exchanges recommendations, builds local predictions, compares behavior, and updates observer-relative trust. Spatial bucketing reduces part of this cost, but the method is still heavier than adaptive and would need further optimization for larger scenarios.

The method also assumes that nearby vehicles are able to communicate and exchange trust information when they meet. This fits the simulation model used here, but real deployment would require assumptions about communication availability, message quality, and compatibility between vehicles.

The results suggest that trust-driven adaptive safety can be implemented in a modular way in microscopic traffic simulation. The

next step is validation on additional scenarios, together with further tuning.

Bibliography

1. HALASZ, David; BUHNOVA, Barbora. *Trust-based Adaptive Safety for Autonomous Systems*. 2024. Unpublished manuscript currently under peer review.
2. HALASZ, David. *Towards Trust-based Adaptive Safety for Autonomous Vehicles*. 2025. Unpublished manuscript currently under peer review.
3. HALASZ, David. From Systems to Ecosystems: Rethinking Adaptive Safety. In: *2022 International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. ACM, 2022, pp. 48–52. Available from DOI: 10.1145/3524844.3528067.
4. HALASZ, David; BUHNOVA, Barbora. Rethinking Safety in Autonomous Ecosystems. In: *Position Papers of the 17th Conference on Computer Science and Intelligence Systems*. ACSIS, 2022, pp. 81–87. ISSN 2300-5963. Available from DOI: 10.15439/2022F273.
5. PLESNÍK, Richard. *Simulation of Fairness Incentivization in Autonomous Ecosystems*. 2024. Thesis. Masaryk University. Unpublished academic source used for ecosystem-oriented simulation methodology.
6. HANČUĽÁK, Radovan. *Adaptive Safety for Autonomous Vehicles Driven by Trust*. 2024. Bachelor's thesis. Masaryk University. Unpublished academic source used as practical inspiration.
7. IQBAL, Danish; BUHNOVA, Barbora. Digital Twin Design for Autonomous Drones. In: *19th Conference on Computer Science and Intelligence Systems, FedCSIS 2024*. Belgrade: IEEE, 2024, pp. 119–130. ISBN 979-8-3503-5971-8. Available from DOI: 10.15439/2024F6765.
8. EUROPEAN TELECOMMUNICATIONS STANDARDS INSTITUTE. *TC ITS: Intelligent Transport Systems* [online]. [visited on 2026-05-18]. Available from: https://www.etsi.org/technical-groups/its/.

9. ALVAREZ LOPEZ, Pablo; BEHRISCH, Michael; BIEKER-WALZ, Laura; ERDMANN, Jakob; FLÖTTERÖD, Yun-Pang; HILBRICH, Robert; LÜCKEN, Leonhard; RUMMEL, Johannes; WAGNER, Peter; WIESSNER, Evamarie. Microscopic Traffic Simulation using SUMO. In: *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2018, pp. 2575–2582. Available from DOI: 10.1109/ITSC.2018.8569938.

10. GERMAN AEROSPACE CENTER (DLR). *SUMO Road Networks – SUMO Documentation* [online]. [visited on 2026-05-18]. Available from: https://sumo.dlr.de/docs/Networks/SUMO_Road_Networks.html.

11. GERMAN AEROSPACE CENTER (DLR). *Definition of Vehicles, Vehicle Types, and Routes – SUMO Documentation* [online]. [visited on 2026-05-18]. Available from: https://sumo.dlr.de/docs/Definition_of_Vehicles%5C%2C_Vehicle_Types%5C%%202C_and_Routes.html.

12. KRAUSS, Stefan. *Microscopic Modeling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics* [online]. 1998. [visited on 2026-05-18]. Available from: https://sumo.dlr.de/pdf/KraussDiss.pdf. PhD thesis. University of Cologne.

13. GERMAN AEROSPACE CENTER (DLR). *Car-Following Models – SUMO Documentation* [online]. [visited on 2026-05-18]. Available from: https://sumo.dlr.de/docs/Car-Following-Models/index.html.

14. GERMAN AEROSPACE CENTER (DLR). *Randomness – SUMO Documentation* [online]. [visited on 2026-05-18]. Available from: https://sumo.dlr.de/docs/Simulation/Randomness.html.

15. GERMAN AEROSPACE CENTER (DLR). *TraCI – SUMO Documentation* [online]. 2025. [visited on 2026-05-10]. Available from: https://sumo.dlr.de/docs/TraCI.html.

16. SIDDIQUI, Sarah Ali; MAHMOOD, Adnan; SHENG, Quan Z.; SUZUKI, Hajime; NI, Wei. Trust in Vehicles: Toward Context-Aware Trust and Attack Resistance for the Internet of Vehicles. *IEEE Transactions on Intelligent Transportation Systems*. 2023, vol. 24, no. 9, pp. 9546–9560. Available from DOI: 10.1109/TITS.2023.3268301.

17. HASTINGS, Erin; MESIT, Jaruwan. Optimization of large-scale, real-time simulations by spatial hashing. 2005.

18. TANGE, Ole. *GNU Parallel 20260322*. Zenodo, 2025-03. Available from DOI: 10.5281/zenodo.19321428. GNU Parallel is a general parallelizer to run multiple serial command line programs in parallel without changing them.

A Electronic Attachments

The complete codebase and the results mentioned in this thesis are available for download in the Information System of Masaryk University. Setup instructions are provided in `README.md`.